
Memory Is a Program: Evolving Agent Memory Through Executable Code Search

Author One
Affiliation
email@example.com

Abstract

LLM agents typically rely on memory systems with fixed architectures chosen at design time, yet the optimal way to store and retrieve information varies across tasks. We introduce ENGRAM, a system that treats agent memory as an executable program and discovers task-optimized memory architectures through evolutionary search over Python code. Each memory program defines how observations are parsed into structured items, how items are stored using a toolkit of relational, vector, and neural primitives, and how relevant context is retrieved for a given query. An LLM-based reflector diagnoses failures on held-out queries and proposes code patches; a compile-fix loop and runtime violation recovery handle the failure modes specific to code evolution. Population-based search with softmax selection maintains structural diversity across candidates. We evaluate ENGRAM on four benchmarks spanning conversational recall, embodied task completion, medical question answering, and professional reasoning. The evolutionary process converges to structurally distinct programs for each task: a multi-signal episodic index for conversational recall, a deterministic action cache for embodied tasks, and criterion-keyed knowledge stores for rubric-graded domains. These results suggest that optimal memory is not a universal design pattern but a task-specific artifact discoverable through automated search.

1 Introduction

Memory is central to how LLM agents accumulate and reuse knowledge across interactions. A conversational assistant must recall facts mentioned hours earlier; an embodied agent must apply lessons from past episodes to novel environments; a medical reasoning system must organize clinical knowledge for accurate triage. Each of these tasks demands a different memory organization: episodic indexes with temporal and speaker metadata, deterministic action caches keyed by task type, or criterion-aligned knowledge stores filtered by domain-specific rubrics.

Existing agent memory systems commit to a fixed architecture at design time. Retrieval-augmented generation stores observations as flat documents and retrieves by embedding similarity [7], discarding relational, temporal, and hierarchical structure. More structured approaches such as G-Memory [19] and Mem0 [19] introduce specific organizational principles (graph-based linking, atomic fact extraction), but each embodies a single design philosophy that may suit some tasks while underperforming on others. Recent work on learnable memory selects among predefined strategies [21, 20] or optimizes natural-language descriptions of memory operations, but these closed design spaces cannot express the computational logic (SQL schemas, stateful aggregations, conditional retrieval) that task-specific memory may require.

We propose ENGRAM, a system that formulates agent memory as an executable Python program and discovers task-optimized architectures through evolutionary search. A memory program defines three evolvable dimensions: *schema* (dataclass fields controlling how observations and queries are parsed),

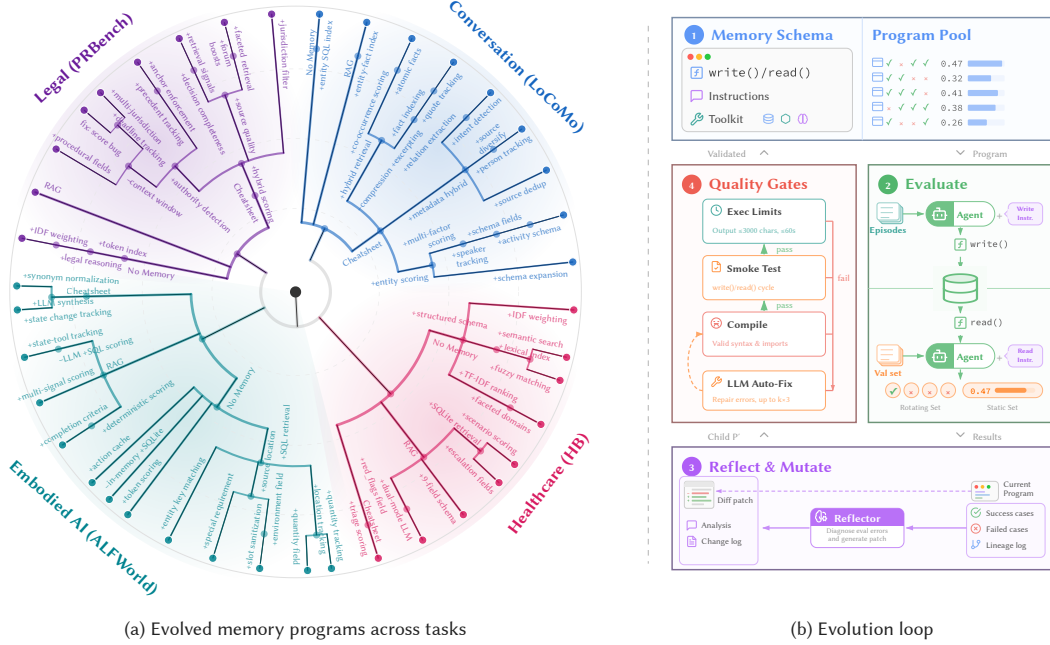


Figure 1: **System overview of ENGRAM.** Each iteration samples a parent program P from the pool via softmax over fitness scores, evaluates it on a split validation set (static subset for scoring, rotating subset for reflection), then an LLM reflector analyzes failed and successful cases alongside the source code, change logs, and pool programs to produce a code patch, yielding a child program P' . Before admission to the pool, the child must pass three quality gates: compilation (AST parsing and import whitelist), a smoke test (dummy `write/read` cycle), and execution limits (≤ 60 s per call, ≤ 3000 character read output); failures are discarded. The memory program defines `write()/read()` logic, instruction constants, and accesses a toolkit of SQLite, ChromaDB, and budget-limited LLM calls.

logic (`write` and `read` methods that store and retrieve knowledge using relational, vector, and neural primitives), and *instructions* (string constants that guide the task agent’s behavior). An LLM-based reflector examines failed evaluation cases and proposes code patches in unified diff format, preserving working logic while targeting specific weaknesses. Because LLM-generated code can fail to compile or violate runtime constraints, we introduce a compile-fix loop and runtime violation recovery that extend reflective evolution from natural-language prompts to executable programs. Population-based search with softmax parent selection maintains structural diversity, enabling the search to discover multiple viable architectures rather than converging prematurely to a single strategy.

We evaluate ENGRAM on four benchmarks that test complementary memory capabilities: LoCoMo (multi-session conversational recall), ALFWorld (embodied task completion from expert demonstrations), HealthBench (medical question answering against professional rubrics), and PRBench (expert-level legal and financial reasoning). Across these benchmarks, the evolutionary process converges to structurally distinct programs that share no architectural similarity, despite starting from the same seed programs and search hyperparameters. On LoCoMo, the best program builds a multi-signal episodic index combining SQL filtering with vector retrieval; on ALFWorld, it constructs a deterministic action cache that retrieves fixed action sequences without invoking the toolkit LLM. HealthBench and PRBench produce criterion-keyed knowledge stores with domain-specific filtering logic.

Our contributions are as follows:

1. We formulate agent memory as an executable program search problem and introduce ENGRAM, which evolves memory architectures through LLM-guided code mutation with compile-fix recovery and population-based search.

2. We evaluate on four benchmarks spanning conversational, embodied, medical, and professional reasoning tasks, comparing against retrieval baselines, five ALMA memory strategies, and the GEPA program-evolution approach.
3. We show that different tasks produce structurally distinct optimal memory programs, providing evidence that memory architecture is a task-specific artifact rather than a universal design choice.

2 Method

We introduce ENGRAM, a system that automatically discovers task-optimized memory programs through evolutionary search over executable code. We first formulate the optimization problem (Section 2.1), then describe the memory program interface that defines the search space (Section 2.2), the reflective mutation operator that proposes program modifications (Section 2.3), and the population-based search strategy that maintains candidate diversity (Section 2.4). Figure 1 provides an overview of the full system.

2.1 Problem Formulation

Consider a task agent $\mathcal{A}$ that processes a sequence of observations $D = \{d_1, \dots, d_N\}$ and answers queries $Q = \{q_1, \dots, q_M\}$. The agent is equipped with a *memory program* P that provides two operations: `write`, which ingests each observation into a knowledge base, and `read`, which retrieves relevant information for a given query. The agent’s prompt and model weights remain fixed throughout; all performance variation is attributed to P . This constraint isolates the optimization signal: any change in task performance directly reflects the quality of the memory program.

Given a task metric $\mu : \mathcal{Y} \times \mathcal{Y} \rightarrow [0, 1]$ that scores the agent’s output against a ground-truth answer, we define the objective as the average metric over all validation queries:

$$J(P) = \frac{1}{M} \sum_{j=1}^M \mu(\mathcal{A}(q_j; P.\text{read}(q_j)), y_j) \quad (1)$$

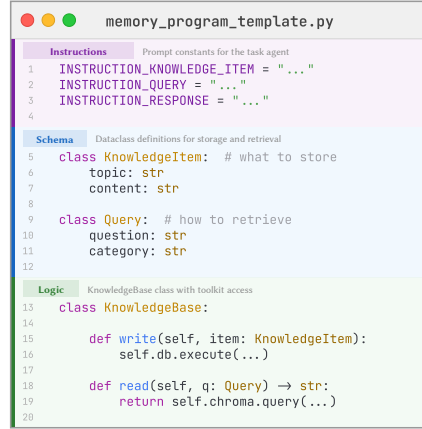
The goal is to find $P^* = \arg \max_P J(P)$ within a budget of B evaluations. Fixing the agent decouples the search from model-level optimization: the metric provides a direct training signal for the memory program alone.

Design space. Unlike prior work that selects among pre-defined memory architectures [20] or optimizes natural-language skill descriptions [21], ENGRAM searches over executable Python programs. This open design space can express arbitrary data structures and deterministic logic, from SQL schemas to hierarchical summaries to state machines.

2.2 Memory Program

A *memory program* is a Python module that defines three dataclasses and four instruction constants (Figure 2). This fixed interface constrains the search to three dimensions: how observations are parsed into structured items, how items are stored and retrieved, and how the task agent is guided through natural-language instructions.

Data classes. `KnowledgeItem` and `Query` are Python dataclasses whose fields define how the task agent parses raw observations and questions, respectively. Each field carries a type annotation restricted to JSON-compatible primitives and a metadata description that is exposed to the



```

memory_program_template.py

Instructions
Prompt constants for the task agent
1 INSTRUCTION_KNOWLEDGE_ITEM = "..."
2 INSTRUCTION_QUERY = "..."
3 INSTRUCTION_RESPONSE = "..."
4

Schema
Dataclass definitions for storage and retrieval
5 class KnowledgeItem: # what to store
6     topic: str
7     content: str
8
9 class Query: # how to retrieve
10    question: str
11    category: str
12

Logic
KnowledgeBase class with toolkit access
13 class KnowledgeBase:
14
15     def write(self, item: KnowledgeItem):
16         self.db.execute(...)
17
18     def read(self, q: Query) -> str:
19         return self.chroma.query(...)
20

```

Figure 2: Memory program interface. A memory program is a Python module with three evolvable dimensions: *instructions* (four string constants injected into the task agent’s prompt), *schema* (dataclass fields defining how observations and queries are parsed), and *logic* (the `write/read` methods that store and retrieve knowledge using the toolkit).

agent as a generation schema. Evolution can reshape these schemas freely: adding or removing fields, changing types, and refining descriptions to steer how the agent extracts information.

KnowledgeBase class. The core of each program is a class with `write` and `read` methods. The `write` method receives both the agent-parsed item and the raw source text, so that the program can trust the agent’s summary, process the raw text with its own logic, or combine both. The `read` method accepts a query object and returns a string of at most 3000 characters, which is injected into the agent’s prompt as retrieved context.

Toolkit. Programs access three resources through a shared toolkit object: (1) an in-memory SQLite database for structured storage and relational queries, (2) an in-memory ChromaDB instance for semantic vector retrieval, and (3) a budget-limited LLM endpoint for neural reasoning within the program itself. These three primitives cover the major memory paradigms (relational, vector, and neural), and evolved programs freely combine them.

Instruction constants. Four module-level string constants control the task agent’s behavior without modifying the agent itself. A *parsing instruction* guides how observations are converted into knowledge items; a *query instruction* governs query construction; a *response instruction* controls the format, length, and style of the agent’s answers; and a *persistent context* string is injected into every agent prompt alongside retrieved memory. These constants are co-evolved with the program’s code, enabling simultaneous optimization of data structures and agent behavior. This co-evolution is important because it allows the system to adapt not only what is stored and retrieved, but also how the agent interprets and responds to retrieved information.

Runtime constraints. Each read call is capped at 3000 characters and 60 seconds; violations cause the corresponding sample to receive a score of zero. These constraints are declared in the reflection prompt so that the evolution process remains aware of the limits.

2.3 Reflective Evolution

Each evolution step selects a parent program from the pool, evaluates it on the task, diagnoses failures through LLM-based reflection, and produces a mutated child program. This procedure is inspired by GEPA’s reflective prompt mutation [1], but operates on executable source code rather than natural-language prompts.

Reflection prompt. The reflector LLM receives a structured prompt containing: (i) the interface specification and runtime constraints, (ii) the current program’s source code, (iii) the evaluation score, (iv) a small number of failed cases with the question, expected answer, model output, and retrieved context, and (v) optionally, successful cases as positive examples. Each component is separated by nested XML tags for clarity. We limit the prompt to two failed cases and one success case by default; in our experience, fewer complete cases produce better reflections than many truncated ones.

Patch-based mutation. The reflector outputs a natural-language diagnosis followed by a code patch in unified diff format. The patch is applied to the parent program’s source code to produce a candidate child. Operating on diffs rather than generating entire programs encourages targeted modifications that preserve already-working logic.

Compile-fix loop. LLM-generated code frequently contains errors that would not arise in prompt evolution. After patch application, the candidate undergoes three validation checks: abstract syntax tree parsing, an import whitelist check, and a smoke test that executes a dummy `write/read` cycle. If any check fails, a dedicated fix prompt presents the error message to the LLM for repair. This loop repeats up to k times (we use $k=3$). The full mutation pipeline is summarized as:

$$P' = \text{COMPILEFIX}^k(\text{APPLYPATCH}(P_{\text{parent}}, \text{REFLECT}(P_{\text{parent}}, \mathcal{R}))) \quad (2)$$

where $\mathcal{R}$ denotes the reflection context (evaluation results and failed cases).

Runtime violation recovery. If a compiled program produces runtime violations during evaluation (e.g., timeouts or output overflow), a dedicated fix step presents the violation details to the reflector, and the program is re-evaluated after each repair attempt. This mechanism extends the compile-fix loop from compilation errors to runtime errors, covering all failure modes specific to code evolution.

Algorithm 1 ENGRAM: Evolutionary Memory Program Search

(a) Evolution Loop

Require: Seed programs $\{S_1, \dots, S_K\}$, budget B , temp. τ

Require: Train data D_{tr} , static val D_s , rotate pool D_r

Ensure: Best program P^*

```
1:  $\mathcal{P} \leftarrow \emptyset$ 
2: for  $k = 1$  to  $K$  do
3:    $(J_k, \_)$   $\leftarrow$  EVAL( $S_k, D_{tr}, D_s$ )
4:    $\mathcal{P} \leftarrow \mathcal{P} \cup \{(S_k, J_k)\}$ 
5: end for
6: for  $t = 1$  to  $B$  do
7:    $P_{par} \leftarrow$  SOFTMAXSAMPLE( $\mathcal{P}, \tau$ )
8:    $\tilde{D} \leftarrow$  SAMPLE( $D_r, n_r$ )
9:    $(\_, \mathcal{R}) \leftarrow$  EVAL( $P_{par}, D_{tr}, \tilde{D}$ )
10:   $P_{mut} \leftarrow$  MUTATE( $P_{par}, \mathcal{R}$ )
11:   $P' \leftarrow$  COMPILEFIXk( $P_{mut}$ )
12:   $(J', \_) \leftarrow$  EVAL( $P', D_{tr}, D_s$ )
13:   $\mathcal{P} \leftarrow \mathcal{P} \cup \{(P', J')\}$  {Unconditional}
14: end for
15: return  $\arg \max_{(P, J) \in \mathcal{P}} J$ 
```

(b) Split Evaluation

Require: Program P , task agent $\mathcal{A}$, metric μ

Require: Train data D_{tr} , eval queries D_{eval}

Ensure: Score J , diagnostic cases $\mathcal{R}$

```
1: // Phase 1: Knowledge ingestion
2:  $KB \leftarrow P.init()$ 
3: for  $d \in D_{tr}$  do
4:    $item \leftarrow \mathcal{A}.extract(d, P.INSTR_{KI})$ 
5:    $KB.write(item, d)$ 
6: end for
7: // Phase 2: Retrieval and scoring
8:  $\mathcal{F}, \mathcal{S} \leftarrow \emptyset, \emptyset$ 
9: for  $(q, y) \in D_{eval}$  do
10:   $query \leftarrow \mathcal{A}.formulate(q, P.INSTR_Q)$ 
11:   $c \leftarrow KB.read(query)$ 
12:   $\hat{y} \leftarrow \mathcal{A}.respond(q, c, P.INSTR_R)$ 
13:   $s \leftarrow \mu(\hat{y}, y)$ ; add  $(q, y, \hat{y}, c)$  to  $\mathcal{F}$  or  $\mathcal{S}$ 
14: end for
15: return  $(\frac{1}{|D_{eval}|} \sum s, (\mathcal{F}, \mathcal{S}))$ 
```

2.4 Population-Based Search

Linear evolution, in which only the latest candidate is mutated, is fragile: a single destructive mutation can derail the entire search trajectory. To mitigate this, we maintain a *program pool* that stores all evaluated candidates, enabling the search to recover from earlier successful programs.

Program pool. The pool $\mathcal{P} = \{(P_i, J_i)\}$ is an unbounded set of (program, score) pairs. Every evaluated child is admitted unconditionally, regardless of whether it outperforms its parent. This design ensures structural diversity within the pool and prevents premature convergence, since even low-scoring programs may contain valuable partial solutions that future mutations can build upon.

Softmax parent selection. At each iteration, a parent is sampled from the pool with probability proportional to an exponentiated score:

$$\Pr(P_i) = \frac{\exp(J_i/\tau)}{\sum_j \exp(J_j/\tau)} \quad (3)$$

where τ is a temperature parameter (we use $\tau=0.15$). Low temperature biases selection toward high-scoring programs while retaining nonzero probability for structurally different weaker candidates. This mechanism is simpler than the Pareto-based selection used in GEPA [1]; we validate its effectiveness in ablation experiments (Section 2).

Multi-seed initialization. The pool is initialized with multiple seed programs implementing distinct strategies, such as experience-based learning, LLM-assisted summarization, and vector retrieval. All seeds are evaluated at iteration zero before evolution begins. These diverse starting points reduce sensitivity to initialization and provide the reflector with a broader set of structural ideas to draw from.

The full procedure is summarized in Algorithm 1. Part (a) shows the evolution loop: at each iteration, the parent is scored on a *static* validation subset D_s for selection, while a freshly sampled *rotating* subset $\tilde{D} \subset D_r$ provides diagnostic cases for reflection. This split prevents the reflector from overfitting to a fixed set of failure cases. Part (b) details the evaluation subroutine: train observations are ingested via `write`, then each eval query is answered via `read` and scored. The algorithm resembles a (1+1) evolution strategy with memory: each iteration produces one child (serial evaluation), but parent selection draws from the full history rather than only the current best. Softmax selection, unconditional admission, and multi-seed initialization together form a simple

population-based search that avoids the complexity of Pareto-based methods [1] while maintaining sufficient exploration.

3 Experimental Setup

We evaluate ENGRAM on four benchmarks that test complementary memory capabilities. For each benchmark, we adopt a three-way data split: train observations are ingested into the knowledge base via `write`; a validation split supplies queries for computing the fitness score $J(P)$ during evolution; and a held-out test split is evaluated once on the best program after evolution terminates. All baselines share the same task agent, model, and evaluation protocol, differing only in the memory program; performance gaps are therefore attributable to the memory strategy alone.

3.1 Benchmarks

LoCoMo [8] tests recall across multi-session conversations. The dataset contains 10 extended dialogues spanning multiple timestamped sessions; the associated questions require single-hop recall, temporal reasoning, causal inference, and multi-hop reasoning across session boundaries. We ingest conversation sessions as train observations and score with token-level F1 [11].

ALFWorld [14] tests embodied task completion in simulated household environments. The agent must accomplish goals (finding, heating, cleaning, and placing objects) through text-based interaction with TextWorld. We ingest expert episode transcripts as train observations; the agent interacts with the live environment at evaluation time using retrieved context as its only source of task-specific knowledge. We report binary success rate within 50 steps and evaluate on both seen and unseen environment splits.

HealthBench [2] tests medical question answering against professional rubrics. Each record provides a multi-turn clinical dialogue, an idealized completion, and a structured rubric of pass/fail criteria authored by medical professionals. We evaluate on two categories: health data interpretation and emergency referral recognition. Performance is measured by rubric-based scoring: each criterion is independently graded by an LLM judge, and the score equals the fraction of positive criteria met.

PRBench tests expert-level professional reasoning in legal analysis and financial valuation. Each record contains a professional task prompt, an expert reference, and an importance-weighted rubric with six levels ranging from critically important (+10) to critically detrimental (−10). We apply the same rubric-based scoring protocol as HealthBench.

Dataset sizes and split configurations are provided in Appendix A.

3.2 Baselines

We organize baselines into three categories by memory paradigm (Table 1).

No Memory provides a lower bound: the task agent answers each query without invoking `write` or `read`.

Retrieval-based systems store raw observations and retrieve by similarity. *Vector Search* stores each observation in a vector collection and retrieves the top- k most similar documents by embedding cosine similarity, with no task-specific parsing or indexing. *G-Memory* [19] adds a hierarchical graph layer that links related tasks via SQLite and combines vector retrieval with graph traversal. *Mem0* [19] extracts atomic facts from observations and maintains them through LLM-driven add/update/delete operations.

Self-evolution systems distill reusable knowledge from experience rather than storing raw episodes. *Trajectory Retrieval* [19] retrieves full episode transcripts by embedding similarity without LLM processing. *ReasoningBank* [?] extracts key insights from each episode via LLM and retrieves them alongside similar task descriptions. *Dynamic Cheatsheet* [19] maintains a single global summary that is rewritten by the LLM after each new experience.

Prompt-optimizing systems improve agent behavior by evolving the program or prompt rather than the stored content. *GEPA* [1] treats the memory program as a text artifact and optimizes it via

Table 1: **Main results.** Performance on LoCoMo (token F1 / LLM judge), ALFWorld (success rate), HealthBench (rubric score), and PRBench (rubric score). Best results per column in **bold**. Best results per column in **bold**.

	LoCoMo		ALFWorld		HealthBench		PRBench	
Method	F1	L-J	Unseen	Seen	Data	Emerg.	Legal	Finance
No Memory	0.036	0.030	0.780	0.640	0.242	0.429	0.431	0.269
<i>Retrieval-based Systems (Memorizing Episodes)</i>								
Vector Search	0.256	0.400	0.700	0.720	0.264	0.400	0.466	0.308
G-Memory	0.224	0.380	0.740	0.560	0.309	0.447	0.499	0.318
Mem0	0.373	0.540	0.780	0.640	0.216	0.413	0.450	0.321
<i>Self-evolution Systems (Memorizing Experiences)</i>								
Trajectory Retrieval	0.276	0.420	0.760	0.780	0.265	0.442	0.480	0.304
ReasoningBank	0.194	0.380	0.780	0.580	0.315	0.470	0.508	0.330
Dynamic Cheatsheet	0.124	0.190	0.680	0.480	0.286	0.487	0.474	0.327
<i>Prompt-optimizing Systems (Memorizing Traction)</i>								
GEPA	0.132	0.190	0.880	0.720	0.304	0.466	0.568	0.449
GEPA + Vector Search	0.300	0.390	0.840	0.820	0.327	0.484	0.554	0.411
ENGRAM	0.459	0.610	0.900	0.700	0.390	0.493	0.660	0.586

LLM reflection with Pareto-efficient candidate selection. We run two variants: GEPA from an empty seed and GEPA initialized from a vector-search seed (*GEPA + Vector Search*).

3.3 Implementation Details

ENGRAM assigns three distinct LLM roles. The *task agent* (GPT-5.4-mini via Azure) parses observations, constructs queries, and generates answers; it remains identical across all conditions including baselines. The *reflector* (GPT-5.3-Codex, reasoning effort medium) diagnoses failures and generates code patches. The *toolkit LLM* (GPT-5.4-mini) is available to evolved programs for neural computation within `write` and `read`, with a per-operation call budget.

We run evolution for 20 iterations per benchmark. The program pool is initialized with three structurally diverse seeds (Section 2.4). Fitness is computed on a validation subset selected via embedding-based clustering; after evolution, the best program is revalidated on the full validation set and evaluated once on the held-out test split. Each evaluation instantiates a fresh knowledge base, ensuring no state leakage across iterations. Additional hyperparameters are provided in Appendix ??.

4 Results and Analysis

We evaluate ENGRAM on four benchmarks (LoCoMo, ALFWorld, HealthBench, and PRBench) under the configurations described in Section 3. Table 1 summarizes the main results. We organize our analysis around four observations that characterize the behavior and outcomes of evolutionary memory program search.

Observation 1: Evolutionary search progressively discovers better memory structures. Table 1 shows that ENGRAM achieves the highest score on seven of eight benchmark configurations. On LoCoMo, the evolved program reaches 0.459 F1, a 23.1% improvement over Mem0 (0.373), the strongest fixed baseline. On ALFWorld unseen, ENGRAM scores 0.900, surpassing GEPA (0.880) and all retrieval-based methods. On ALFWorld seen, ENGRAM reaches 0.700, below GEPA+Vector Search (0.820); the seen split contains environments encountered during GEPA’s prompt optimization, giving prompt-level adaptation an advantage over code-level structural changes on familiar tasks. The gains are largest on PRBench, where ENGRAM scores 0.660 on legal (+29.9% over ReasoningBank) and 0.586 on finance (+77.6% over ReasoningBank), suggesting that rubric-based professional reasoning benefits most from task-adapted memory structure. On HealthBench, ENGRAM improves on the best baseline by 23.8% on data tasks (0.390 vs. 0.315) and reaches 0.493 on emergency referrals, comparable to Dynamic Cheatsheet (0.487).

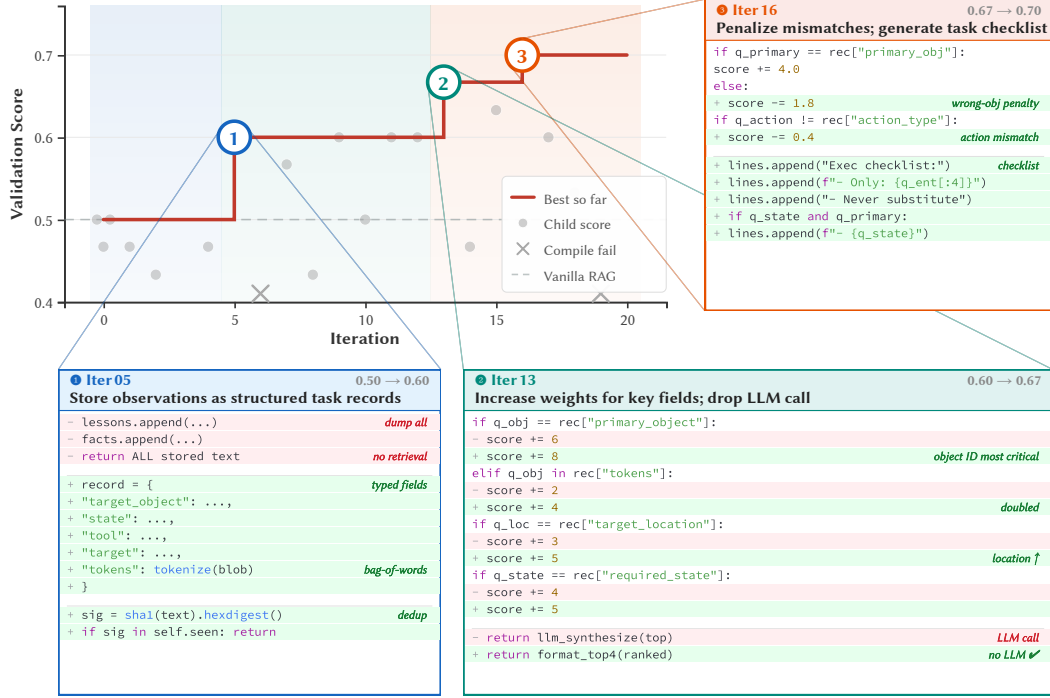


Figure 3: **Evolution trajectory.** Validation score across iterations for all benchmarks. Early iterations correct structural errors in seed programs; middle iterations discover task-relevant indexing strategies with the largest gains; later iterations refine retrieval precision with diminishing returns.

Figure 3 traces the validation score across evolution iterations for all benchmarks. The trajectory reveals distinct phases: early iterations correct structural errors inherited from seed programs, middle iterations discover task-relevant indexing strategies that produce the largest score gains, and later iterations refine retrieval precision with diminishing returns. This phased progression suggests that the search process first establishes a viable program skeleton and then optimizes its internal logic.

Observation 2: Different tasks produce structurally distinct optimal memory programs. The best programs evolved for LoCoMo and ALFWorld differ not only in parameter values but in their fundamental data structures, storage strategies, and retrieval logic. On LoCoMo, the best program implements a *multi-signal episodic index*: it parses each conversation session into structured records with speaker, timestamp, and topic fields stored in an SQL table, indexes the raw text in a vector collection for semantic retrieval, and combines relational filtering with embedding similarity at read time to assemble context that spans multiple sessions. On ALFWorld, the best program implements a *deterministic action cache*: it extracts task-type and subgoal patterns from expert demonstrations, stores them in a lookup table keyed by task category, and retrieves a fixed action sequence template at read time without invoking the toolkit LLM.

Figure 4 contrasts these two programs. The same evolutionary system, initialized with the same three seed programs and the same search hyperparameters, converges to architectures that share no structural similarity. This divergence is not an artifact of random variation; it reflects the fact that conversational recall and procedural task completion impose fundamentally different demands on memory organization. LoCoMo rewards flexible, multi-signal retrieval over declarative content, while ALFWorld rewards deterministic, low-latency lookup of procedural knowledge.

HealthBench and PRBench provide further evidence of this structural divergence. Both benchmarks require an agent to satisfy fine-grained rubric criteria in professional domains (clinical triage and legal or financial reasoning, respectively), placing a premium on memory programs that organize domain-specific knowledge into structured, criterion-aligned representations rather than general episodic indexes. The programs evolved for these benchmarks introduce yet another class

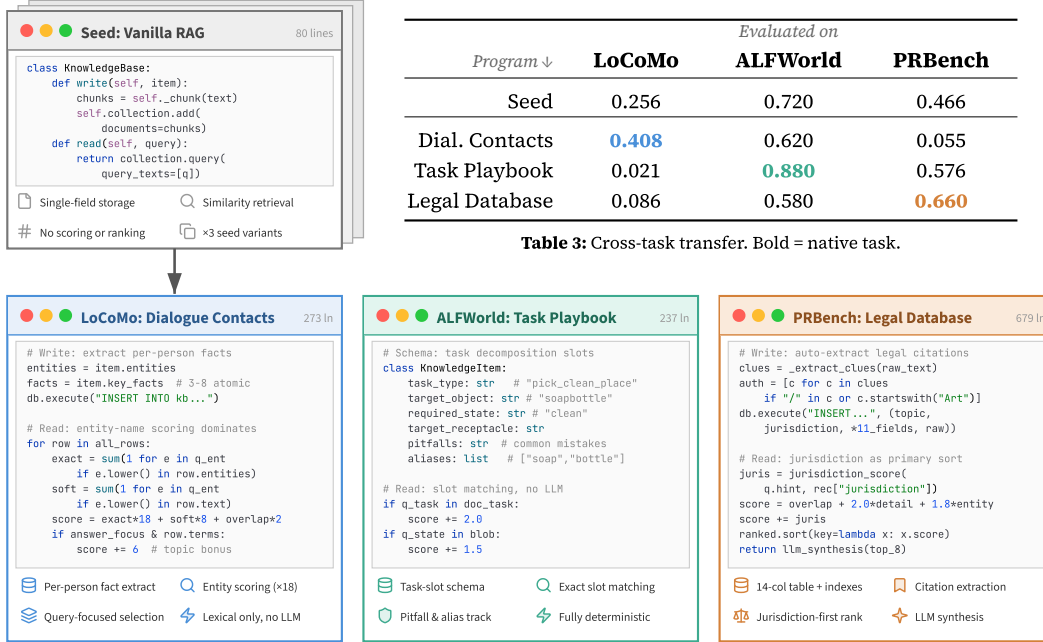


Figure 4: **Task-optimized memory programs** evolved for LoCoMo (left, blue) and ALFWorld (right, red), starting from the same Vector Search seed (top-left). The same evolutionary system with identical seeds produces structurally distinct architectures. **LoCoMo** evolves a *multi-signal episodic index*: 8-field declarative records are stored in an SQL table, retrieved via 7-component lexical scoring with intent-aware boosts, and synthesized by an LLM call. **ALFWorld** evolves a *deterministic action cache*: task-type slots are inferred via 15+ regex patterns, records are scored by typed field matching with wrong-object penalties, and output is a formatted execution checklist with no LLM in the retrieval path. The cross-task transfer matrix (top-right) shows performance when swapping evolved code structures and instruction policies between tasks, confirming that structure and policy must be co-optimized.

of architecture—criterion-keyed knowledge stores with domain-specific filtering logic—that shares neither the relational schema of the LoCoMo solution nor the action-sequence cache of the ALFWorld solution. Taken across all four benchmarks, each task class produces a structurally distinct optimal program, reinforcing the conclusion that memory architecture is a task-specific artifact rather than a universal design choice.

These results support the central thesis of this work: optimal memory is not a universal design pattern but a task-specific artifact. Hand-designing a single memory architecture and applying it across tasks necessarily sacrifices performance on tasks whose structure does not match the designer’s assumptions. Automated search over an open design space sidesteps this problem by allowing the structure itself to adapt.

Observation 3: Joint evolution of structure and policy enables task-specific adaptation. Observation 2 shows that different tasks produce different optimal programs; we now examine the mechanism. Table 2 reports the contribution of each design choice through ablation experiments on LoCoMo. Removing code evolution (– Code structure) causes the largest drop: F1 falls to 0.256, a 44.2% relative decrease, confirming that structural adaptation of data structures and retrieval logic is the primary driver of performance. Re-

Table 2: **Ablation study** on LoCoMo. Each variant removes one design choice from the full system. Δ : absolute change; Rel.%: relative change. Δ : absolute change from full system; Rel.%: relative change.

Variant	F1	Δ	Rel.%
Full system	0.459	—	—
– Instruction constants	0.353	–0.106	–23.1
– Code structure	0.256	–0.203	–44.2
– Population search	0.381	–0.078	–17.0
– Population diversity	0.318	–0.141	–30.7

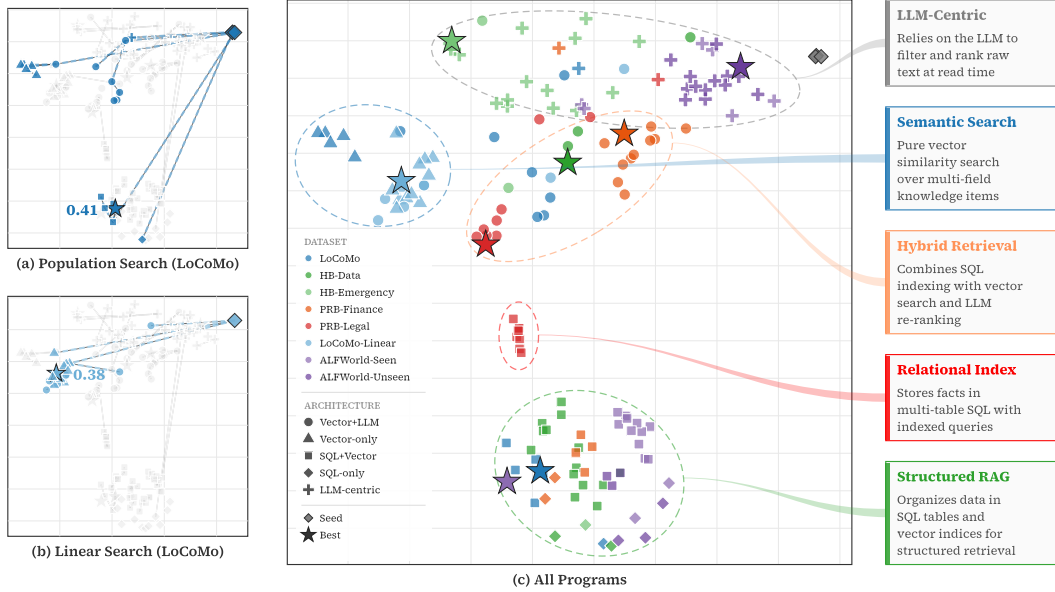


Figure 5: **Program embedding landscape.** Each evolved program is embedded with a code embedding model and projected to 2D via t-SNE. **(a, b)** Population-based search (Engram) explores structurally diverse regions of the program space, while linear search concentrates in a narrow neighborhood. Colored edges trace parent – child lineage to shared seed programs. **(c)** All programs across five benchmarks, colored by dataset. Marker shapes indicate the storage architecture discovered by evolution: circles for vector + LLM retrieval, triangles for vector-only, squares for SQL + vector, rotated squares for SQL-only, and crosses for LLM-centric designs. Dashed ellipses denote emergent architectural clusters. Distinct benchmarks converge on different clusters, confirming that optimal memory architecture is task-dependent.

moving population-based search (– Population search) reduces F1 to 0.381 (–17.0%), indicating that the ability to revisit and branch from earlier programs provides a modest but consistent benefit. Freezing the four instruction constants (– Instruction constants) reduces F1 to 0.353 (–23.1%), showing that co-evolving instructions with code structure provides a meaningful benefit: the instruction constants adapt extraction and retrieval prompts to complement the evolved data structures.

Observation 4: The search landscape contains multiple structurally diverse optima. Figure 5 visualizes the program embedding landscape. The program pool accumulated during evolution contains multiple high-scoring programs that implement distinct strategies. On LoCoMo, the top five programs by validation score include variants that emphasize temporal filtering, speaker-based partitioning, and topic clustering, each achieving scores within 0.00 of the pool maximum. On ALFWorld, high-scoring programs split between action-sequence caching and subgoal-decomposition strategies.

This diversity is not an artifact of noise; programs with similar scores employ fundamentally different data structures and retrieval logic. The population-based search discovers these alternatives because softmax selection occasionally revisits lower-scoring parents whose structural lineage diverges from the current best. Linear evolution, which always mutates the most recent candidate, cannot recover these alternative branches once they are abandoned (Figure 5a vs. b). The existence of multiple viable optima suggests that the fitness landscape over memory programs is rugged, with several peaks separated by regions of low fitness that require structural changes to traverse.

Taken together, these observations demonstrate that optimal memory is not a universal design pattern but a task-specific artifact discoverable through automated search over executable programs. The

evolutionary process does not converge to a single best architecture; it reveals a landscape of diverse solutions shaped by task demands.

5 Related Work

5.1 Memory for LLM Agents

Prior work on agent memory focuses on constructing external memories from interaction histories and leveraging them to support downstream reasoning and decision making [22]. Typical pipelines periodically extract salient information into a memory store, retrieve relevant entries for a new query, and update the store via consolidation or pruning [10, 24, 16]. More recently, MemSkill [21] frames memory operations as learnable natural-language skills selected by an RL controller, and AWM [17] induces reusable workflows from successful trajectories to guide future retrieval. Because these approaches commit to a fixed representation at design time, they cannot express task-specific computational logic such as relational schemas or stateful aggregations.

Several concurrent works explore self-evolving memory, but within closed design spaces: Evo-Memory [18] benchmarks test-time memory evolution, and MemEvolve [20] searches over combinations of four predefined modules (encode, store, retrieve, manage), limiting reachable architectures to those already in the library. By contrast, we search over executable Python programs that implement arbitrary data structures and retrieval logic, subsuming the closed spaces of prior systems.

5.2 Self-Evolving Agents

Recent work on self-evolving LLM agents studies how agents can improve from interaction experience without gradient-based parameter updates [5]. ExpeL [23] distills successful and failed trajectories into editable natural-language insights retrieved at test time, and FLEX [3] formalizes this as forward learning, constructing an experience library through continual reflection on environmental feedback. A complementary line automates agent design: ADAS [6] searches over agentic components via meta-optimization, Voyager [15] synthesizes reusable skill programs in open-ended environments, and Reflexion [13] feeds verbal self-critique back into the next trial. All of these evolve text-level artifacts that an LLM must re-interpret at runtime; ENGRAM instead evolves deterministic code, removing interpretation variance and enabling structures that natural language cannot express.

5.3 LLM-Guided Program Search

Using LLMs as mutation operators for program synthesis has produced solutions surpassing human-designed heuristics in combinatorial optimization [12], neural architecture search [4], and broader algorithmic discovery [9]. Closest to our work, GEPA [1] iteratively mutates every prompt within a compound AI system by reflecting on sampled trajectories in natural language, maintaining a Pareto front to diversify strategies across training instances. ENGRAM applies the same reflective evolution paradigm to executable memory programs; because generated code can fail to compile or violate runtime constraints, we introduce a compile-fix loop and runtime violation recovery that are unnecessary in prompt evolution (Section 2.3).

6 Conclusion

We presented ENGRAM, a system that discovers task-optimized agent memory by evolving executable Python programs through LLM-guided search. By searching over an open design space of code rather than selecting among fixed architectures, ENGRAM adapts memory structure, retrieval logic, and agent instructions jointly to each task. Experiments on four benchmarks show that the evolutionary process converges to structurally distinct programs: multi-signal episodic indexes for conversational recall, deterministic action caches for embodied tasks, and criterion-keyed stores for rubric-graded professional domains. These results suggest that no single memory architecture is universally optimal; instead, the best organization depends on the task’s demands on storage, retrieval, and reasoning. A limitation of the current approach is the computational cost of evolution, which requires repeated program evaluation across iterations. Future work may explore transfer of evolved programs across related tasks and more sample-efficient search strategies that reduce the number of evaluations required.

References

- [1] Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. *ICLR*, 2026. Oral. [arXiv:2507.19457](#).
- [2] Rahul K. Arora, Jason Wei, Rebecca Soskin Hicks, Preston Bowman, Joaquin Quiñonero-Candela, Foivos Tsimpourlas, Michael Sharman, Meghan Shah, Andrea Vallone, Alex Beutel, Johannes Heidecke, and Karan Singhal. HealthBench: Evaluating large language models towards improved human health. *arXiv preprint arXiv:2505.08775*, 2025.
- [3] Zhicheng Cai, Xinyuan Guo, Yu Pei, Jiangtao Feng, Jiangjie Chen, Ya-Qin Zhang, Wei-Ying Ma, Mingxuan Wang, and Hao Zhou. FLEX: Continuous agent evolution via forward learning from experience. *arXiv preprint arXiv:2511.06449*, 2025.
- [4] Angelica Chen, David Dohan, and David So. EvoPrompting: Language models for code-level neural architecture search. *NeurIPS*, 2023.
- [5] Huan-ang Gao, Jiayi Geng, Wenye Hua, Mengkang Hu, Xinzhe Juan, Hongzhang Liu, Shilong Liu, Jiahao Qiu, Xuan Qi, Yiran Wu, et al. A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence. *arXiv preprint arXiv:2507.21046*, 2025.
- [6] Shengran Hu, Cong Lu, and Jeff Clune. ADAS: Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*, 2024.
- [7] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*, 2020.
- [8] Adyasha Maharana, Dong-Ho Lee, Sergey Tuber, and Mohit Bansal. LoCoMo: Long-context conversation understanding with comprehensive memory. In *ACL*, 2024.
- [9] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.
- [10] Joon Sung Park, Joseph C O’Brien, Carrie J Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *UIST*, 2023.
- [11] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD: 100,000+ questions for machine comprehension of text. In *EMNLP*, 2016.
- [12] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M Pawan Kumar, Emilien Dupont, Francisco JR Ruiz, Jordan S Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024.
- [13] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023.
- [14] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. ALFWorld: Aligning text and embodied environments for interactive learning. In *ICLR*, 2021.
- [15] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.

- [16] Weizhi Wang, Li Dong, Hao Cheng, Xiaodong Liu, Xifeng Yan, Jianfeng Gao, and Furu Wei. Augmenting language models with long-term memory. *arXiv preprint arXiv:2306.07174*, 2024.
- [17] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. *arXiv preprint arXiv:2409.07429*, 2024.
- [18] Tianxin Wei, Noveen Sachdeva, Benjamin Coleman, Zhankui He, Yuanchen Bei, Xuying Ning, Mengting Ai, Yunzhe Li, Jingrui He, Ed H. Chi, Chi Wang, Shuo Chen, Fernando Pereira, Wang-Cheng Kang, and Derek Zhiyuan Cheng. Evo-Memory: Benchmarking LLM agent test-time learning with self-evolving memory. *arXiv preprint arXiv:2511.20857*, 2025.
- [19] Yiming Xiong, Shengran Hu, and Jeff Clune. Learning to continually learn via meta-learning agentic memory designs. *arXiv preprint arXiv:2602.07755*, 2026.
- [20] Guibin Zhang, Haotian Ren, Chong Zhan, Zhenhong Zhou, Junhao Wang, He Zhu, Wangchunshu Zhou, and Shuicheng Yan. MemEvolve: Meta-evolution of agent memory systems. *arXiv preprint arXiv:2512.18746*, 2025.
- [21] Haozhen Zhang, Quanyu Long, Jianzhu Bao, Tao Feng, Weizhi Zhang, Haodong Yue, and Wenya Wang. MemSkill: Learning and evolving memory skills for self-evolving agents. *arXiv preprint arXiv:2602.02474*, 2026.
- [22] Zeyu Zhang, Xiaohe Zhang, Yuanpei Wang, Shengjie Yan, and Rui Sun. A survey on the memory mechanism of large language model based agents. *arXiv preprint arXiv:2404.13501*, 2024.
- [23] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM agents are experiential learners. *AAAI*, 2024.
- [24] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. MemoryBank: Enhancing large language models with long-term memory. *arXiv preprint arXiv:2305.10250*, 2024.

A Dataset Details

Table 3 summarizes the data splits for each benchmark configuration.

Table 3: Dataset split sizes. LoCoMo trains on conversation sessions and evaluates on QA pairs; ALFWorld trains on expert trajectories and evaluates on interactive episodes; HealthBench and PRBench use homogeneous items throughout.

Benchmark	Split	Train	Val	Test
LoCoMo	—	272 sessions	1,440 QA	100 QA
ALFWorld	Unseen	4,652 traj.	145 ep.	50 ep.
	Seen	4,652 traj.	150 ep.	50 ep.
HealthBench	Data Tasks	257	120	100
	Emergency	260	122	100
PRBench	Legal	100	100	50
	Finance	120	130	50

For LoCoMo, we exclude category 5 (adversarial and unanswerable questions), which tests refusal capability rather than memory retrieval, retaining 1,540 QA pairs across four categories. For PRBench, hard items (`finance_hard` and `legal_hard`) are excluded from evolution to avoid distortion of the fitness signal.

B Computational Cost

Table 4 reports the computational cost of running ENGRAM evolution and baselines across all benchmarks. We break down API calls by role: the *task agent* (knowledge extraction, query

generation, and answer generation), the *reflector* (code mutation via LLM reflection), the *judge* (per-criterion rubric scoring for HealthBench and PRBench), and the *toolkit* (LLM calls made by the evolved KB program’s `read()`/`write()` methods). All experiments use gpt-5.4-mini as the task agent, judge, and toolkit model, and gpt-5.3-codex as the reflector model. Cost estimates use Azure OpenAI pricing: \$0.75/\$4.50 per 1M input/output tokens for gpt-5.4-mini, and \$1.75/\$14.00 per 1M input/output tokens for gpt-5.3-codex. Note that gpt-5.3-codex is a reasoning model; its output token counts include internal reasoning tokens, which are billed at the same rate as visible output tokens.

Table 4: Computational cost of 20-iteration evolution runs and baselines. Wall-clock time includes environment interaction overhead (e.g., TextWorld episodes for ALFWorld).

Benchmark	Config	Iter.	Task	Refl.	Judge	TKit	In Tok	Out Tok	Cost	Time
LoCoMo	ENGRAM	20	5802	48	—	135	4.9M	4.2M	\$25.43	5.1h
	Vector Search	0	200	0	—	272	294K	71K	\$0.54	2m
	No Memory	0	200	0	—	272	219K	64K	\$0.45	1m
ALFWorld (Unseen)	ENGRAM	20	3355	25	—	443	7.5M	1.4M	\$13.92	100h
	Vector Search	0	80	0	—	497	711K	50K	\$0.76	14m
	No Memory	0	80	0	—	497	708K	50K	\$0.76	11m
ALFWorld (Seen)	ENGRAM	20	2420	28	—	494	5.6M	1.1M	\$11.94	101h
	Vector Search	0	80	0	—	497	692K	49K	\$0.74	11m
	No Memory	0	50	0	—	224	10M	25K	\$0.31	0m
HB-Emergency	ENGRAM	20	4272	31	21370	2219	21M	6.1M	\$47.20	4.3h
	Vector Search	0	45	0	1102	423	830K	145K	\$1.28	6m
	No Memory	0	54	0	1102	412	761K	137K	\$1.19	1m
HB-DataTasks	ENGRAM	20	4204	24	15006	1575	18M	6.3M	\$44.64	2.1h
	Vector Search	0	24	0	1097	433	950K	177K	\$1.51	7m
	No Memory	0	34	0	1105	423	836K	171K	\$1.40	2m
PR-Legal	ENGRAM	20	3065	28	19984	1242	48M	11M	\$90.19	15h
	Vector Search	0	15	0	958	185	818K	130K	\$1.20	6m
	No Memory	0	15	0	950	185	712K	117K	\$1.06	1m
PR-Finance	ENGRAM	20	1795	31	18602	2497	47M	10M	\$84.81	2.6h
	Vector Search	0	8	0	845	212	740K	153K	\$1.24	6m
	No Memory	0	13	0	895	207	646K	139K	\$1.11	11m

The total evolution cost ranges from \$12–90 per benchmark over 20 iterations, depending primarily on the evaluation protocol. Rubric-graded benchmarks (HealthBench and PRBench) are substantially more expensive because each validation item requires multiple LLM judge calls for per-criterion scoring, making the judge the dominant cost component (60–80% of total calls). Token-F1 benchmarks (LoCoMo) and binary-success benchmarks (ALFWorld) avoid judge calls entirely, keeping evolution under \$26. ALFWorld’s long wall-clock time (100h) is dominated by TextWorld environment interaction, not API latency. Although the reflector makes only 24–48 calls per run, it uses the more expensive gpt-5.3-codex model (\$14/1M output tokens vs. \$4.50 for gpt-5.4-mini) and produces long code patches, so it accounts for 7–24% of total cost depending on the benchmark.

Compared to baselines, the evolution overhead is a one-time cost: once the best program is found, inference uses the same number of calls as any baseline. The per-query inference cost of an evolved program equals that of the corresponding baseline configuration (No Memory or Vector Search) plus any toolkit calls the evolved program makes, typically 0–2 additional LLM calls per query.

C Stability Across Seeds

To assess the robustness of the evolutionary search, we run ENGRAM with three different evolution seeds (42, 1, 2) on three benchmarks spanning different evaluation protocols: LoCoMo (token F1), HealthBench Data Tasks (rubric scoring), and PRBench Finance (rubric scoring). All runs use identical hyperparameters and seed programs; only the random seed controlling parent selection and validation subset rotation differs. Table 5 reports the results.

Table 5: **Stability across evolution seeds.** Test scores for three independent runs per benchmark, each using a different evolution seed. Best baseline: strongest non-evolution method from Table 1. All seeds consistently outperform baselines. CV%: coefficient of variation (lower is more stable).

Benchmark	Test Score by Seed			Mean $\pm$ Std	CV%	Best Baseline
	Seed 42	Seed 1	Seed 2			
LoCoMo (F1)	0.459	0.463	0.364	0.429 ± 0.055	12.9	0.373
HB Data Tasks	0.390	0.382	0.357	0.376 ± 0.017	4.6	0.315
PR Finance	—	—	—	—	—	0.330

[SCAFFOLD] Fill PR Finance scores when runs complete. Update mean/std/CV accordingly. Consider adding: (1) Iter₉₀ convergence speed, (2) pairwise code similarity, (3) strategy match analysis.